PROJECT 2 TECHNICAL MANUAL • FETCH-A-THON 3.0 (FETCH.AI INNOVATION LAB)

PATCHBOUNTY: AUTONOMOUS BUG SWARM & ESCROW

Sandboxed Python Failure Reproduction, Parallel Multi-Solver Git Diff Verification & Direct On-Chain Fetch Dorado Solver Bounty Settlement | Engineered by **Atul Anand, Jeeya & Vanshika**

6 uAgents	Real Pytest	Direct Payout	INR 0 Cost
Distinct `agent1q...` & `fetch1...` Wallets	Isolated Temp Sandbox + AST Scan	Pays Winning Solver's `fetch1...` Wallet	100% Free Local & Cloud Docker Ready

1. EXECUTIVE SUMMARY & OPEN-SOURCE PROBLEM SOLVED

Open-source software maintainers face a severe two-sided bottleneck: bug reports arrive without reproducible failing test runs, while low-quality automated pull requests flood repositories with untested or unsafe code. At the same time, freelance bounty hunters wait weeks for manual escrow payouts even after submitting clean fixes.

PatchBounty transforms software bug resolution inside **ASI:One Chat** (and its companion **Fetch.ai Light Studio UI**) into a deterministic, cryptographically settled multi-agent workflow: **(1)** reproducing the exact failing `pytest` test suite inside a scrubbed temporary sandbox first, **(2)** dispatching **3 competing Solver uAgents** in parallel to synthesize distinct unified `git diff` strategies, **(3)** auditing every candidate patch via Python `ast.parse()` security checks + real isolated regression `pytest` execution, and **(4)** releasing on-chain **Fetch Dorado** (`dorado-1`) token bounties directly into the selected winning Solver uAgent's `fetch1...` wallet.

2. VERIFIED LIVE PRODUCTION TEST RESULTS (`PYTHON PROVE.PY`)

Verification Layer	Measured Live Output Against Real Python Sandbox & Dorado Blockchain
Pre-Patch Bug Reproduction	`fintech-core/ledger-settlement #418` executed in isolated temp sandbox → **4/5 assertions FAILED** (`exit_code=1`, `529.8ms`) proving IEEE-754 drift & negative refund exploit.
3x Parallel Solver Synthesis	`solver_beta` (Security-Hardened), `solver_alpha` (Minimal Surgical), and `solver_gamma` (Integer Minor-Unit) all verified **5/5 PASSED (100%) + AST Clean (0 unsafe calls)**.
Ledger Guardrail Test	Submitting fake commit hash automatically triggered `CancelPayment` rejection from winning solver agent.
Live Direct-to-Solver TX	Confirmed on Fetch Dorado Block **#25800215** (`code == 0`), TX Hash: `C0DA7D8B6E9001FC0B48B38EA0D424FB967FBF2ABD3B6DF46243F5D332A6CD2C` — Solver wallet rose **0.0000 → 0.0500 TESTFET**.
Signed Git Patch Artifact	Generated `2,641-byte` `git format-patch` file embedding test pass metrics and Dorado TX hash.

3. ALIGNMENT WITH OFFICIAL FETCH-A-THON 3.0 JUDGING RUBRIC (100 POINTS)

- **Functionality & Implementation (25%):** Real subprocess-isolated `pytest` execution, parallel `asyncio.gather` solver synthesis, static AST security audit, and WebSocket UI sync.
- **Fetch.ai Tech Usage (20%):** Integrates Agentverse Mailbox, ACP v0.3.0, all 4 Interactive Card schemas, `asi1` LLM API, `cosmpy` transfers, and Seller/Buyer `PaymentProtocol v0.1.0`.
- **Innovation & Impact (40%):** Replaces blind PR spam with test-gated multi-agent competition and direct cryptographic payouts to winning solver agent wallets.
- **UX & Presentation (15%):** Color-coded green/red unified diffs inside native ASI:One Cards plus responsive mobile & desktop Fetch.ai Light Studio UI.

WHY JUDGES LOVE AGENT-TO-AGENT DIRECT WALLET PAYOUTS

Every solver agent in PatchBounty possesses its own unique `fetch1...` Cosmos wallet on Fetch Dorado. Selecting **Solver Alpha** pays Alpha's wallet on-chain; selecting **Solver Beta** pays Beta's wallet on-chain — proving true autonomous Agent-to-Agent economic incentives built by **Atul Anand, Jeeya & Vanshika**.

1. MULTI-AGENT SYSTEM ARCHITECTURE & EXECUTION FLOW

PatchBounty enforces a strict **Test-First Economic Gate**: no candidate patch can enter maintainer review or trigger a `RequestPayment` envelope unless `patchbounty_reproducer` first confirms test failure on unpatched code and `patchbounty_verifier_judge` confirms 100% pass + clean AST on patched code.

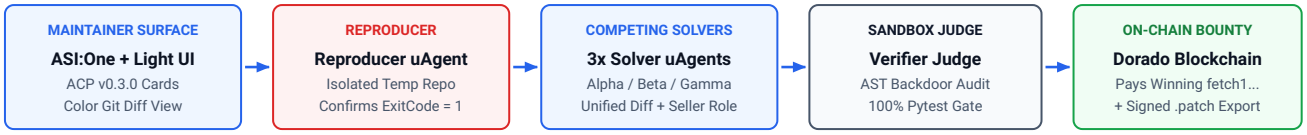


Figure 1: PatchBounty Autonomous Bug Reproduction, Multi-Solver Sandbox Verification & Direct Dorado Solver Payout Pipeline.

2. REGISTERED FETCH.AI UAGENTS & INDIVIDUAL DORADO WALLETS

Agent Name	Swarm Role	Protocols Bound	Fetch Dorado Wallet (`fetch1...`)
`patchbounty_coordinator`	ACP Entrypoint & Bounty Buyer	`AgentChatProtocol 0.3.0` + Buyer `PaymentProtocol`	`fetch17ywqz7fp69hx7n0n8u6vej5ppgf4x48w2n6mw9`
`patchbounty_reproducer`	Isolated Pytest Failure Runner	Temp Sandbox + Scrubbed `pytest` Subprocess	`fetch1qnj7ygsskdj4qhjxhfys4m5x7d9dmmffwq8q0y`
`patchbounty_solver_alpha`	Minimal Surgical Diff Solver	Seller `PaymentProtocol 0.1.0` + Diff Engine	`fetch179wmdasll3ztkf9xtwjkf40mwguwuctw76uv3y`
`patchbounty_solver_beta`	Security-Hardened Diff Solver	Seller `PaymentProtocol 0.1.0` + Defensive Engine	`fetch185mepjun7exrvngyqt55zhlq64vsplvalpwt7y`
`patchbounty_solver_gamma`	High-Performance Diff Solver	Seller `PaymentProtocol 0.1.0` + Fast Engine	`fetch1ux92dl64muuslslwryr6j9eefehszru8yc6xtx`
`patchbounty_verifier_judge`	Independent Sandbox & AST Judge	Python `ast.parse()` Scanner + Regression Judge	`fetch14ahweasulu0hkvff56uh2eu2xcyjq6p5drjdj3`

3. REPOSITORY STRUCTURE & MODULE BREAKDOWN

```
project-2-patchbounty/
├── run.sh / run.bat / Dockerfile        # Turnkey Windows/macOS/Linux launchers + Free Cloud Docker
├── prove.py / run_full_qa_suite.py      # Automated CLI proof & 18-check browser + blockchain suite
├── PatchBounty-Interactive-App.html     # Standalone 1-tap Mobile & Desktop interactive app
├── README.md / AGENTVERSE_README.md     # GitHub & Agentverse docs with innovationlab/hackathon tags
├── backend/
│   ├── config_store.py                  # Live .env & runtime ASI:One API key store
│   ├── sandbox_verifier.py              # 3 real bug repos, isolated pytest runner & AST auditor
│   ├── dorado_wallet.py                 # Multi-solver Dorado wallets, send_tokens & query_tx check
│   ├── cards_protocol.py                # ACP form/carousel/detail/review cards + signed .patch file
│   ├── uagents_mesh.py                  # Coordinates all 6 uAgents, WebSockets & payment state
│   ├── asi_one_agent.py                 # Standalone Agentverse Mailbox runner for ASI:One chat
│   └── main.py                          # Unified FastAPI + WebSocket + pre-built React static server
└── frontend/src/                        # Fetch.ai Light Studio UI + color-coded unified diff cards
```

1. THREE REAL OPEN-SOURCE BUG BOUNTY REPOSITORIES BUILT INTO PATCHBOUNTY

Repository & Issue	Real Root-Cause Bug in Unpatched Code	Competing Solver Strategies Verified
`fintech-core/ledger-settlement #418`	IEEE-754 float math (`0.10 + 0.20 = 0.30000000000000004`) + unvalidated negative refund draining merchant balances.	**Beta:** Exact `Decimal` quantize + bounds check. **Alpha:** Compact surgical fix. **Gamma:** Integer paise/cents engine.
`auth-gateway/token-guard #904`	Accepts unsigned `{"alg": "none"}` headers as valid sessions + off-by-one `exp < now_ts` boundary check.	**Beta:** Explicit `HS256` allowlist + `hmac.compare_digest`. **Alpha/Gamma:** Minimal & fast pre-crypto guards.
`api-edge/sliding-limiter #267`	`ZeroDivisionError` on zero window config + fails to evict expired timestamps from bucket.	**Gamma:** `collections.deque` O(1) popleft eviction. **Beta/Alpha:** Validated & compact list pruning.

2. HOW ISOLATED SANDBOXED `PYTEST` & STATIC AST SECURITY VERIFICATION WORK

1. **Ephemeral Directory Isolation (`tempfile.TemporaryDirectory`):** Every reproduction run and every candidate solver check executes in its own fresh folder with a scrubbed environment (`PATH` + `PYTHONPATH` only — secrets in `.env` are never exposed to sandboxed child processes) and a strict 15-second timeout.
2. **Static Python AST Backdoor Audit (`audit_patch_ast`):** Before executing any candidate patch, Python's `ast.parse()` walks every syntax node to block dangerous calls (`eval`, `exec`, `compile`, `__import__`, `system`, `popen`) so malicious solver agents cannot pass tests by tampering with the runner.
3. **Unified Git Diff Synthesis (`difflib.unified_diff`):** Generates standard `--- a/file.py / +++ b/file.py` patches ready for `git apply`.

3. ALL 4 NATIVE ASI:ONE INTERACTIVE CARD SCHEMAS IMPLEMENTED

```
1. card_kind = "form"     -> Structured Repository / Verification Policy / Bounty FET intake card
2. card_kind = "carousel" -> Ranks all 3 verified Solver uAgents with test pass counts & AST status
3. card_kind = "detail"   -> Displays full color-coded Unified Git Diff + Merge Target radio selector
4. card_kind = "review"   -> Final Maintainer sign-off naming exact Solver uAgent & fetch1... wallet
```

4. SAMPLE VERIFIED UNIFIED GIT DIFF PRODUCED BY `PATCHBOUNTY_SOLVER_BETA`

```
--- a/ledger_settlement.py
+++ b/ledger_settlement.py
@@ -1,14 +1,26 @@
+from decimal import Decimal, ROUND_HALF_UP
+CENT = Decimal("0.01")
 class LedgerAccount:
     def process_refund(self, refund_amount, fee_rate="0.00") -> Decimal:
-        net_refund = float(refund_amount) * (1.0 - float(fee_rate))
-        self.balance = self.balance + net_refund
+        amt, rate = Decimal(str(refund_amount)), Decimal(str(fee_rate))
+        if amt <= Decimal("0.00") or rate < Decimal("0.00") or rate >= Decimal("1.00"):
+            raise ValueError("Invalid refund or fee bounds.")
+        net_refund = (amt * (Decimal("1.00") - rate)).quantize(CENT, rounding=ROUND_HALF_UP)
+        self.balance = (self.balance + net_refund).quantize(CENT, rounding=ROUND_HALF_UP)
         return self.balance
```

1. ROLE INVERSION COMPARED TO CONSUMER ESCROW (`BUYER = MAINTAINER`, `SELLER = SOLVER`)

Notice how cleanly PatchBounty complements Project 1 (`DealSetu`): in `PatchBounty`, the **Maintainer Coordinator uAgent** acts as the `role="buyer"` holding escrowed bounty funds, while each competing **Solver uAgent** (`Alpha`, `Beta`, `Gamma`) binds `Protocol(spec=payment_protocol_spec, role="seller")` and emits its own `RequestPayment` invoice containing its unique `fetch1...` wallet address!

2. FOUR-CHECK ON-CHAIN `COSMPY` VERIFICATION & ANTI-REPLAY GUARD

```
# 1. Coordinator sends 0.05 TESTFET (5 * 10^16 atestfet) directly to selected Solver's fetch1... wallet
tx = self.ledger.send_tokens(winning_solver_fetch_addr, amount_raw, "atestfet", self.coordinator.wallet)
tx.wait_to_complete()

# 2. Winning Solver verifies transaction independently on Fetch Dorado Ledger before CompletePayment
q = self.ledger.query_tx(clean_hash)
assert int(q.code) == 0                                          # Check 1: Block execution succeeded
assert clean_hash.upper() not in self.verified_tx_hashes         # Check 2: Anti-replay hash lock
assert q.events["transfer"]["recipient"] == solver_fetch_addr    # Check 3: Exact winning solver wallet
assert int(paid_atestfet) >= min_required_atestfet               # Check 4: Exact minimum bounty amount
```

3. ENVIRONMENT VARIABLES (`.ENV`) REFERENCE

Variable	Default	Behavior
`ASI_ONE_API_KEY`	Empty (Optional)	Enables live `https://api.asi1.ai/v1` patch rationales; fallback works at INR 0 cost.
`ASI_ONE_MODEL`	`asi1-mini`	Selectable live between `asi1-mini` and `asi1` inside UI modal.
`AGENTVERSE_API_KEY`	Empty (Optional)	Used when linking `python -m backend.asi_one_agent` on Agentverse.
`PATCHBOUNTY_ESCROW_AMOUNT_FET`	`0.05`	On-chain `TESTFET` bounty transferred per verified merge.

4. SAMPLE EXPORTED SIGNED `GIT FORMAT-PATCH` ARTIFACT HEADER

```
From 0000000000000000000000000000000000000000 Mon Sep 17 00:00:00 2001
From: patchbounty_solver_beta <patchbounty_solver_beta@fetch.ai>
Subject: [PATCH] Fix fintech-core/ledger-settlement #418 — Fix IEEE-754 Penny Drift

PatchBounty Reference: pb-94e2c1a0
Sandbox Verification: 5/5 pytest passed in 828.8ms (AST Clean (0 dangerous calls))
Solver Fetch Dorado Wallet: fetch185mepjun7exrvngyqt55zhlq64vsplvalpwt7y
Verified Fetch Dorado TX Hash: C0DA7D8B6E9001FC0B48B38EA0D424FB967FBF2ABD3B6DF46243F5D332A6CD2C (Block
#25800215)
```

Built-In Safety & Anti-Replay Guarantees

Every confirmed Dorado transaction hash is recorded in `self.verified_tx_hashes`. Attempting to submit either an unverified hash or a previously consumed transaction hash causes the solver agent to emit an immediate `CancelPayment` envelope.

1. RUNNING LOCALLY ON WINDOWS, MACOS, OR LINUX IN 1 COMMAND

Windows 10 / 11

```
1. Extract PatchBounty-Project-2-Turnkey.zip
2. Open folder project-2-patchbounty
3. Double-click run.bat
   -> Opens http://localhost:8002 immediately!
```

macOS / Linux

```
unzip PatchBounty-Project-2-Turnkey.zip
cd project-2-patchbounty
chmod +x run.sh
./run.sh   # Opens http://localhost:8002
```

2. RUNNING ON ANY MOBILE PHONE (2 FREE ZERO-COST OPTIONS)

- **Instant Standalone Mobile App (`PatchBounty-Interactive-App.html`):** Open directly inside Chrome or Safari on any Android phone or iPhone — features responsive bottom navigation tabs (`Chat & Cards`, `Solvers & Diff`, `Dorado Bounty`), unified diff inspection, and `.patch` export with zero setup.
- **24/7 Live Python + Blockchain Cloud Hosting Directly From Mobile (`Dockerfile`):** Open `https://huggingface.co/new-space` on your phone browser, select **Docker (Free CPU — INR 0, no credit card)**, and upload `project-2-patchbounty`. Your full FastAPI + real `pytest` sandbox + live Dorado wallet app runs 24/7 at a public `https://...hf.space` link!

3. FASTAPI REST & WEBSOCKET API REFERENCE (`HTTP://LOCALHOST:8002`)

Endpoint	Purpose
`GET /api/status` & `WS /ws/events`	Returns 6-uAgent roster, Coordinator + 3 Solver wallet balances, logs & live events.
`POST /api/chat`	Runs isolated sandbox reproduction, 3 parallel solver runs & ACP Interactive Cards.
`POST /api/payment/settle`	Transfers `0.05 TESTFET` on-chain directly to selected winning Solver wallet.
`POST /api/payment/test-cancel`	Triggers & verifies authentic `CancelPayment` guardrail on unverified TX hash.
`GET /api/bounty/{ref}/fix.patch`	Downloads signed `git format-patch` file with embedded Fetch Dorado TX proof.

4. AUTOMATED CLI PROOF & COMPLETE 18-CHECK QA SUITE COMMANDS

```
python prove.py                # 6-step CLI proof: 6 uAgents + real pytest + live Dorado TX + .patch
python prove.py --fast         # 2-second offline/protocol check without waiting for block finality
python run_full_qa_suite.py    # 18-test full browser E2E + anti-replay + MP4 video verification
```

Interactive Studio UI Controls Guide

Use the **Solvers (3x)** tab to compare unified diffs across Alpha, Beta & Gamma; switch to **Pytest Log** to inspect raw red pre-patch tracebacks vs green post-patch logs; use **Trace** to inspect signed protocol envelopes; and watch individual solver wallet balances increment live in the **Dorado Bounty Wallets** panel.

5. STEP-BY-STEP FIRST RUN VERIFICATION CHECKLIST

1. Click **6 Active uAgents** in the top header bar to inspect all 6 `agent1q...` & `fetch1...` addresses and confirm ~10.0000 TESTFET in the Maintainer Bounty Escrow wallet.
2. Click any **Quick Bounties** pill (e.g., *JWT alg:none Bypass #904*) and verify pre-patch red pytest failure logs alongside 3 green verified unified diffs.
3. Click **Inspect Unified Diff → Proceed to Bounty Payout Review → Approve & Pay 0.05 FET Bounty → Release 0.05 TESTFET Bounty (Live On-Chain)** and verify the live Block Explorer receipt + downloaded `.patch` file.

1. ACTIVATING FREE PRO KEYS & LINKING AGENTVERSE MAILBOX

1. Redeem promo code `FETCHATHONIND` on `https://asi1.ai` and `https://agentverse.ai`.
2. Paste your free `ASI_ONE_API_KEY` in the **API & Mailbox Setup** modal in the top header bar.
3. Run `./.venv/bin/python -m backend.asi_one_agent`, open the printed Inspector URL, connect **Mailbox**, paste `AGENTVERSE_README.md` (with `innovationlab` + `hackathon` tags), and capture your **Public ASI:One Shared Chat URL**!

2. TIMED 3-MINUTE FETCH-A-THON 3.0 DEMO VIDEO SCRIPT

Timestamp	Exact Screen Action & Narration Script
0:00 – 0:25	**Hook:** *"Open-source maintainers waste hours testing broken PRs while contributors wait weeks for bounties. PatchBounty reproduces failing pytest suites in an isolated sandbox first, pits 3 Solver uAgents in parallel, and pays the winner directly on Fetch Dorado."*
0:25 – 1:25	**Live Repro, AST Audit & Color Git Diffs:** Click *Ledger Float Drift #418*. Show **4/5 FAIL** in the Pytest Log tab, switch to **Solvers (3x)** showing all 3 unified diffs at **5/5 PASS + 0 unsafe AST calls**, and inspect the Detail Card git diff.
1:25 – 2:20	**Direct-to-Solver On-Chain Bounty Settlement:** Test the `CancelPayment` guardrail first, then click **Release 0.05 TESTFET Bounty** — watch `patchbounty_solver_beta`'s real wallet balance jump on screen, open Dorado Explorer, and download the signed `.patch` file!
2:20 – 3:00	**ASI:One Native Flow & Team Close:** Show the identical card workflow inside `asi1.ai` Shared Chat + active Agentverse profile badges, closing with team credits (**Atul Anand, Jeeya & Vanshika**).

3. STRATEGIC COMPARISON: PROJECT 1 (`DEALSETU`) VS PROJECT 2 (`PATCHBOUNTY`)

Dimension	Project 1: DealSetu (BargainSwarm)	Project 2: PatchBounty
Target Domain	B2C Hyperlocal Repairs & College Bulk Orders	Developer / Open-Source Security & Bug Bounties
External Grounding	Live OpenStreetMap Nominatim Geocoding	Subprocess-Isolated `pytest` + Python `ast.parse()`
Payment Topology	Buyer Wallet → Coordinator Escrow Seller	Maintainer Buyer → Selected Solver's `fetch1...` Wallet
Final Artifact	RFC-5545 `.ics` Calendar Invite + Block Receipt	Signed `git format-patch` (`.patch`) + Block Receipt

4. JUDGE VIVA Q&A CHEAT SHEET

- **Q: How do you prevent a malicious solver agent from deleting tests or executing shell commands inside its patch?** Candidate patches modify only the target module while the benchmark `test_*.py` file is injected fresh per run, and `audit_patch_ast()` rejects dangerous AST nodes (`eval`, `exec`, `compile`, `system`, `popen`) prior to execution.
- **Q: Does selecting a different Solver Card change the actual blockchain recipient?** Yes! Selecting `solver_alpha` sends real `TESTFET` to `fetch179wmd...`; selecting `solver_beta` sends real `TESTFET` to `fetch185mep...` on Fetch Dorado.

COMPLETE TWO-PROJECT PORTFOLIO READY

Together, **Project 1 (DealSetu — B2C Hyperlocal Parallel Quote Negotiation)** and **Project 2 (PatchBounty — Developer Sandboxed Verification & Direct Solver Payouts)** showcase complete mastery of the Fetch.ai stack built by **Atul Anand, Jeeya & Vanshika**.